

Collision Detection Science

Research Reference, Pure-Rust Porting Evaluation & the `posim` Implementation

`physical_object_simulator`

July 24, 2026

This is the typeset twin of `collision_detection.md` — companion to `grammar.pdf`, `physical_object_simulator` and `scene_info.pdf`. Written for a reader with zero prior knowledge.

Contents

1	What is rigid-body collision detection?	2
2	The research: seven simulators investigated	2
2.1	Bullet / PyBullet (C++, the robotics standard)	2
2.2	Rapier / parry (pure Rust, the modern reference)	2
2.3	Project Chrono / PyChrono (C++, multi-physics engineering)	3
2.4	SOFA (C++, interactive multi-physics)	3
2.5	Open Source Physics / EJS (Java, the teaching classic)	3
2.6	SIMple Physics (Rust; engine = <code>nphysics/ncollide</code>)	3
2.7	PhysicsHub (JavaScript, student web sims)	3
3	The comparison chart	3
3.1	The concept tree (technique $\rightarrow$ source $\rightarrow$ <code>posim</code> decision)	4
4	Evaluation for a pure-Rust port (zero unsafe, zero dependencies)	5
5	The recommendation: collision detection in ALL rigid-body scenes	6
6	The <code>posim</code> implementation	6
7	Command reference	9
8	How it was verified	11
9	The twelve examples (run for the USER and the MANAGER)	12
9.1	Example 1 — head-on exchange (the canonical action–reaction pair)	13
9.2	Example 2 — unequal masses (the light ball bounces back)	13
9.3	Example 3 — the restitution ladder	13
9.4	Example 4 — Newton’s cradle (five spheres)	13
9.5	Example 5 — billiard break (every outgoing direction is a normal)	13
9.6	Example 6 — off-center spin-up ($L = r \times J\hat{n}$)	14

9.7	Example 7 — the thin-wall time of impact (the tunneling killer)	14
9.8	Example 8 — colliding gravitational binary	14
9.9	Example 9 — the spinning target ($\omega \times r$ terms at work)	14
9.10	Example 10 — the billiard box (seven bounces, zero drift)	14
9.11	Example 11 — the box of shapes (every shape in the infinitely massive box)	15
9.12	Example 12 — two dumbbells from a user-defined function — E , P and L conserved	15

10	FAQ	17
----	-----	----

1 What is rigid-body collision detection?

Two solid bodies flying through space must not pass through each other. A simulator therefore answers three questions, every time step:

1. **Did anything touch?** (*detection*) — and, harder: did anything touch *during* the step, even if the step’s endpoints look separated?
2. **Where and in which direction?** (*contact geometry*) — the **contact point** and the **contact normal** $\hat{n}$, the unit vector along which Newton’s third law acts: body j receives the impulse $+J\hat{n}$ and body i receives exactly $-J\hat{n}$. Get the normal wrong and momentum leaks; get it right and the force pair is a true action–reaction pair.
3. **What happens next?** (*response*) — an impulse (instantaneous momentum exchange), a constraint force, or a stiff penalty spring.

The hard parts, discovered independently by every engine surveyed: **tunneling** (a fast body crosses a thin one between two checks), **resting contact** (an infinite sequence of ever-smaller bounces — the Zeno problem), **simultaneity** (three balls touching at once — Newton’s cradle), and **penetration recovery** (bodies that already overlap).

2 The research: seven simulators investigated

2.1 Bullet / PyBullet (C++, the robotics standard)

Broad phase: `btDbvtBroadphase` — *two* dynamic AABB trees (moving vs. static/sleeping); alternatives `btAxisSweep3` (sweep-and-prune) and brute force. Narrow phase: GJK (`btGjkPairDetector`) + **EPA** penetration depth, with analytic special cases for sphere-sphere, sphere-box, box-box (SAT). Manifold: `btPersistentManifold`, ≤ 4 cached points, persisted across frames (**warm starting**), pruned by a breaking threshold (≈ 0.02). Normal convention: `contactNormalOnB`, from B toward A; `contactDistance` < 0 = penetration. CCD: swept-sphere conservative advancement + **motion clamping**. Response: `btSequentialImpulseConstraintSolver` (projected Gauss–Seidel), defaults 10 iterations, `erp` 0.2, **split impulse** below -0.04 , restitution suppressed under a resting-velocity threshold, warm-start 0.85, margin 0.04. Islands, sleeping, SIMD.

2.2 Rapier / parry (pure Rust, the modern reference)

Broad phase: BVH over collider AABBs (quantized SIMD-friendly `Qbvh`). Narrow phase: parry GJK + penetration, SAT primitives; `ContactManifolds` carry per-shape local normals + a world normal, per-point `dist` < 0 = penetration, persisted solver impulses. Normal: from collider 1

toward collider 2. CCD: **nonlinear time-of-impact** (translation and rotation) + motion clamping. Response: sequential impulse with stabilization iterations, SoA/SIMD batches, restitution threshold, islands + sleeping.

2.3 Project Chrono / PyChrono (C++, multi-physics engineering)

Collision systems: embedded Bullet, or the Multicore system (spatial binning + analytic/MPR). Physics formulations: **NSC** (non-smooth — contacts as hard constraints, a Differential Variational Inequality solved as a Cone Complementarity Problem by **APGD**) vs. **SMC** (penalty/compliant, Hertz-style). **Envelope & margin**: contacts anticipated at an outward envelope before touching. Normal: the X column of the per-contact `plane_coord` rotation matrix. No dedicated CCD.

2.4 SOFA (C++, interactive multi-physics)

Pipeline: `BruteForceBroadPhase` → `BVHNarrowPhase` → pluggable intersection methods with **alarmDistance** (contact created *before* overlap — the anti-tunneling band) and **contactDistance** (the separation the response holds). `DetectionOutput` carries both points, a normal (outward from the first model) and a signed distance (< 0 = interpenetration). Response: penalty springs or Lagrangian unilateral constraints (Gauss–Seidel).

2.5 Open Source Physics / EJS (Java, the teaching classic)

Analytic 2-body overlap tests (center distance vs. sum of radii); the response in closed form — velocity decomposed along the **line of centers** (the normal), along-normal components transformed by the 1-D elastic formulas. Teaching devices: impact-parameter slider, the center-of-mass cross sailing straight through the collision.

2.6 SIMple Physics (Rust; engine = nphysics/ncollide)

`ncollide` (ancestor of `parry`): DBVT broad phase with loosened AABBs, GJK + EPA, **persistent manifolds** with feature IDs, proximity events. `SIMple` adds a user-editable Lua `collision_fn` — students see and modify the response formula (line-of-centers impulse; a `cos_angle < 0` guard against re-colliding separating pairs).

2.7 PhysicsHub (JavaScript, student web sims)

Squared-distance reject, exact circle overlap, Wikipedia two-body elastic formula along the center line, a damping slider as restitution, live KE plot. Fixed-timestep overlap testing — the honest example of *how tunneling happens* when nothing guards against it.

3 The comparison chart

(Split across two tables; same rows.)

	Bullet	Rapier/parry	Chrono	SOFA
Broad phase	dual DBVT / SAP	QbvH BVH	DBVT or grid binning	brute-force AABB
Narrow phase	GJK+EPA, SAT prims	GJK+pen., SAT prims	GJK/EPA or analytic+MPR	BVH + proximity
Manifold	persistent ≤ 4 pts	persistent + impulses	Bullet's / own	DetectionOutput list
Normal	on B, B $\rightarrow$ A	collider1 $\rightarrow$ collider2	X col of plane_coord	outward from 1st
Penetration	dist < 0	dist < 0	dist < 0	dist < 0
TOI / anti-tunnel	swept-sphere + clamp	nonlinear TOI + clamp	envelope + small dt	alarmDistance band
Response	seq. impulse PGS	seq. impulse	NSC (APGD) or SMC	penalty or Lagrangian
Stabilization	split impulse, erp 0.2	bias iterations	constraint/compliance	contactDistance
Restitution	resting-vel threshold	velocity threshold	material pairs	material
Key numbers	margin .04, break .02	substeps, margins	envelope/margin	alarm/contact dist

	OSP / EJS	ncollide (SIMple)	PhysicsHub
Broad phase	none (2-body)	DBVT (loose AABBs)	squared-distance reject
Narrow phase	analytic circles	GJK+EPA	analytic circles
Manifold	single implicit	persistent + feature IDs	none
Normal	line of centers	GJK/EPA geometric	line of centers
Penetration	overlap bool	depth ≥ 0	overlap bool
TOI / anti-tunnel	exact 2-body algebra	swept TOI	none (tunnels!)
Response	closed-form elastic	impulse/constraint	closed-form + damping
Stabilization	exact	slop + projection	none
Restitution	e in formula	material	damping slider

3.1 The concept tree (technique $\rightarrow$ source $\rightarrow$ posim decision)

```

collision detection science
|-- DETECTION
|   |-- broad phase (find candidate pairs cheaply)
|   |   |-- brute-force AABB cull      <- SOFA, PhysicsHub  -> ADOPTED (O(n^2))
|   |   |-- sweep-and-prune            <- Bullet             -> upgrade path
|   |   |-- dynamic BVH / QbvH / binning <- Bullet/Rapier/Chrono -> upgrade path
|   |-- narrow phase (exact pair geometry)
|   |   |-- analytic primitive pairs    <- OSP lineage       -> ADOPTED
|   |   |-- SAT for boxes (15 axes)     <- Bullet box-box   -> ADOPTED
|   |   |-- GJK + EPA (any convex)      <- Bullet, parry     -> NOT needed (Sec. 4)
|   |-- contact geometry
|   |   |-- contact point                <- all                -> ADOPTED
|   |   |-- UNIT NORMAL (action-reaction) <- all                -> ADOPTED: from i to j
|   |   |-- signed separation / depth    <- all (dist < 0)    -> ADOPTED; ALSO the sundials root function
|   |-- 'time of impact (anti-tunneling) * THE HARD PROBLEM *
|   |   |-- swept shapes + clamping     <- Bullet, Rapier   -> surveyed, not needed
|   |   |-- proximity band              <- SOFA, Chrono     -> surveyed
|   |-- 'INTEGRATOR EVENT ROOTFINDING <- (none of the 7!) -> ADOPTED:

```

```

|
|                                     CNodeRootInit lands on the exact root
|-- RESPONSE
| |-- impulse (Gauss-Seidel passes)   <- Bullet, Rapier   -> ADOPTED (K-matrix)
| |-- restitution e + velocity thresh. <- Bullet, Rapier   -> ADOPTED (e=1, min)
| |-- positional projection + slop     <- Bullet split imp. -> ADOPTED
| |-- Lagrangian / APGD                <- Chrono NSC, SOFA  -> overkill
| |-- penalty springs                  <- SOFA, Chrono SMC  -> rejected (stiffness
|                                     fights adaptive CNODE)
'-- HURDLES (and the posim answer)
    |-- tunneling                      -> rootfinding + CNodeSetMaxStep cap (verified: 100 m/s
    |                                     vs 5 mm plate)
    |-- resting contact/Zeno -> tiered guard: 64 events -> plastic; 128 -> disarm+
    project
    |-- simultaneity                    -> Gauss-Seidel over all pairs (Newton's cradle emerges)
    |-- deep initial overlap -> end-of-interval sweep + projection
    '-- degenerate geometry -> skipped safely, never a division by zero

```

4 Evaluation for a pure-Rust port (zero unsafe, zero dependencies)

Essential — adopted: AABB broad-phase cull; analytic narrow phase (with a *closed* shape set {point, sphere, cuboid} every pair has an exact closed form — faster than GJK, no termination heuristics, and the normal is **exact**, which is the entire point); SAT for cuboid-cuboid; the impulse response with the effective-mass matrix K (provably momentum-conserving action–reaction along $\hat{n}$, angular terms give correct spin transfer); restitution with a velocity threshold; positional projection with slop; sequential Gauss–Seidel multi-pair passes; and — *novel here* — **integrator event rootfinding for the time of impact**: the integration backend (SUNDIALS) already contains a root finder that checks every internal step and interpolates onto the crossing, a precision no engine-side swept test can match.

Valuable — documented upgrade tier: persistent manifolds with warm starting (stacking); 2–4-point face manifolds (boxes resting flat); sweep-and-prune/BVH broad phase (hundreds of bodies); Coulomb friction (per-object μ , tangential impulse clamped to the cone — designed, deliberately not shipped in this delivery); islands + sleeping.

Overkill — rejected with reasons: GJK/EPA/MPR (no open shape set to serve); NSC complementarity solvers (APGD — research-grade machinery for granular matter); penalty springs (their stiffness would fight CVODE’s adaptive error control); GPU/SIMD batching (no workload to feed it).

Why no GJK/EPA (the one decision an expert would question): GJK exists to handle *arbitrary* convex shapes behind a support function. posim’s shape set is closed, so all six pair types have exact constant-time formulas with exact normals; Bullet itself special-cases exactly these pairs. If a general convex hull is ever added, GJK+EPA (or parry) becomes the right tool; the **Contact** record and the solver do not change. *(The shape set has since grown — torus, disk, cylinder, and now the compound dumbbell — and remains closed: Section 6’s three-tier dispatch keeps every ball contact and every face contact exact, still without GJK; the dumbbell never even reaches the dispatch whole — it decomposes into ball and cylinder parts first.)*

5 The recommendation: collision detection in ALL rigid-body scenes

1. **On by default, everywhere.** Every STEP, RUN and scene-window playback detects collisions whenever a collidable pair exists; COLLIDE OFF opts out. Two Points cannot collide; two static bodies are skipped.
2. **Event-driven time of impact.** The pairwise **signed separation** is the integrator's root function (CvodeRootInit / ARKodeRootInit). CVODE/ARKODE check it after every internal step and interpolate onto the zero crossing: measured impact-time errors are at the 10^{-15} relative level. Approach-only direction filtering plus the integrator's zero-at-restart handling prevent re-triggering.
3. **Anti-tunneling cap.** While armed, the internal step is capped at $(\text{smallest feature}) / (2 \times \max \text{reachable relative surface speed})$ — center speed plus what the accelerations can add before the next refresh, plus each body's spin bound times R_{bound} , since a rotating edge can sweep into contact without the centers moving — refreshed after every event and at every interval start (Section 6 has the exact bounds).
4. **Exact impulse at the root.** $J = -(1 + e)(\mathbf{v}_{\text{rel}} \cdot \hat{n}) / (\hat{n}^\top K \hat{n})$ with $K = (m_i^{-1} + m_j^{-1}) \mathbf{1} - [r_i]_\times I_{i,w}^{-1} [r_i]_\times - [r_j]_\times I_{j,w}^{-1} [r_j]_\times$, applied as $\pm J \hat{n}$ (and $\pm r \times J \hat{n}$) through the canonical setters — the action–reaction pair by construction. Static bodies receive no writes. $e = \min(e_i, e_j)$, default 1.
5. **Guards.** Tiered Zeno guard (64 events $\rightarrow$ plastic; 128 $\rightarrow$ disarm + project); *elastic* end-of-interval penetration sweep (deep initial overlaps and grinding approximate contacts); degenerate geometry skipped safely.
6. **The normal is public.** Grammar paths (`contact0.normal`), CONTACTS, machine-mode JSON, and golden arrows in the scene window.
7. **Structural safety.** With zero collidable pairs the rootfinding is never armed and the CVODE code path is **bit-identical** to the pre-collision implementation (verified in the sundials source and enforced by test).

6 The posim implementation

Modules. `physical_object/src/boundary.rs` (the shape set: SDFs, support extents/points/ranks, analytic inertia); `physical_object/src/collide.rs` (geometry + impulse response, fully unit-tested); event wiring in `physical_object/src/integrate.rs` (CVODE and SPRK event loops); grammar in `posim` (COLLIDE, CONTACTS, BOX, `contactK` paths); scene arrows + shape wireframes in `posim/src/scene/`.

The shape set. Seven boundaries: Point, Sphere, Cuboid, the shapes-and-box release's Torus (a centerline circle of radius c in the local xy -plane swept by a tube of radius a ; inner radius $c - a$, outer radius $c + a$), Disk (an **ideal zero-thickness** solid disk in the local xy -plane) and Cylinder (radius r , half-height h ; full height $2h$), and — since this release — the first **compound** shape, Dumbbell{`r1`, `r2`, `rod_radius`, `z1`, `z2`, `f1`, `f2`}: solid sphere 1 (radius r_1 , mass fraction f_1) centered at $(0, 0, z_1)$, solid sphere 2 (r_2, f_2) at $(0, 0, z_2)$, joined by a solid rod of radius `rod_radius` — two spheres plus a rod as **one** rigid body. The `boundary::dumbbell(m1, m2, m_rod, r1,`

`r2`, `rod_radius`, `length`) constructor computes the offsets from the part masses — $z_1 = -(m_2 + m_{\text{rod}}/2)L/M$, $z_2 = (m_1 + m_{\text{rod}}/2)L/M$ (L the center-to-center length, $M = m_1 + m_2 + m_{\text{rod}}$) — so the local origin **is** the center of mass: the identity $m_1 z_1 + m_2 z_2 + m_{\text{rod}}(z_1 + z_2)/2 = 0$ holds exactly (pinned by test), and the stored fractions f_1/f_2 keep every part mass recoverable from the total. Each non-legacy shape carries an **exact SDF** (torus: distance to the centerline circle minus the tube radius; disk: the unsigned distance, zero exactly on the disk — a zero-thickness body has no interior; cylinder: the 2-D box SDF in (ρ, z) ; dumbbell: the exact **union** — the min of its parts' SDFs, two spheres and the rod's capped cylinder) and the **analytic inertia tensor**: torus $I_z = m(c^2 + \frac{3}{4}a^2)$, $I_x = I_y = m(\frac{1}{2}c^2 + \frac{5}{8}a^2)$; disk $I_x = I_y = \frac{1}{4}ma^2$, $I_z = \frac{1}{2}ma^2$ (perpendicular-axis theorem); cylinder $I_z = \frac{1}{2}mr^2$, $I_x = I_y = m(3r^2 + 4h^2)/12$; dumbbell the exact **composite** — $\frac{2}{5}mr^2$ for each sphere plus its parallel-axis term mz^2 , plus the rod's cylinder terms about the COM (hand-checked). Every shape also provides its **support extent** in exact closed form — since this release the **true directed** support $h(u) = \max_x x \cdot u$: the dumbbell is the first non-centrally-symmetric shape (its off-center spheres make $h(u) \neq h(-u)$), and every symmetric shape's closed form is bit-identically unchanged (for them $h(-u) = h(u)$); `world_aabb` now takes the $\pm$ directed extents per axis, and for the torus the support is that of its convex hull — a support function cannot see the hole. Each shape also answers a **support point** achieving the extent — the **centroid of the supporting set** when that set is a face, edge, circle or cap, so a flat-on contact carries no spurious lever arm — and a **support rank** (0 = single point/rim point, 1 = edge or circle, 2 = face or cap).

The Contact record (pinned convention, ARCHITECTURE.md §3.8):

```
pub struct Contact {
    pub i: usize,      // first body (lower index)
    pub j: usize,      // second body
    pub t: f64,        // event time (the interpolated root)
    pub point: Vec3,   // world-space contact point
    pub normal: Vec3,  // UNIT normal, from body i toward body j
    pub depth: f64,    // penetration depth >= 0 (~ 0 at a root)
    pub rel_vel_n: f64, // pre-impulse (v_j - v_i).n (< 0 = approaching)
    pub impulse_n: f64, // scalar impulse magnitude J
}
```

The three-tier dispatch (still no GJK/EPA/MPR machinery):

1. **Exact ball tier** — a *ball* (sphere, or point as the zero-radius case) against **anything**: separation = the partner shape's exact SDF at the ball center (in the shape frame) minus the ball radius. Sphere-sphere keeps $|\Delta| - (r_a + r_b)$ with the normal along the line of centers; sphere-cuboid keeps the box-SDF formula (continuous inside and out; the interior branch picks the nearest face). For torus/disk/cylinder the contact is the exact closest surface point, the normal the exact outward direction, and the contact point sits midway across the overlap band. Because the SDFs are exact, a small ball **can genuinely thread a torus hole** (its separation never reaches zero on a through-hole line) and a point particle **passes through the ideal zero-thickness disk** (a measure-zero contact — see the FAQ). Degenerate configurations (ball center on the torus centerline, exactly on the disk, on the cylinder axis at side contact) are skipped safely.
2. **Exact cuboid SAT** — cuboid-cuboid is the 15-axis SAT, **unchanged**: the maximum separation is the signed separation, its axis (oriented $i \rightarrow j$) the normal; contact point =

deepest support vertex (face case) or the closest points of the supporting edges (edge case).

3. **Support-axis tier** — every remaining extended-vs-extended pair (any pair involving a torus/disk/cylinder not covered above; a dumbbell is decomposed first — see below — so only its rod, as a free cylinder, can arrive here). The **directed** SAT gap $d \cdot l - h_a(l) - h_b(-l)$ is evaluated for **both orientations** of every candidate axis — for centrally symmetric shapes $h(-u) = h(u)$ and it reduces exactly to the classic $|d \cdot l| - (e_a + e_b)$; the dumbbell’s off-center spheres need the general form — over a finite candidate-axis list: each cuboid’s three **face axes**, each round shape’s **symmetry axis**, every normalized **cross product** of those primary axes (edge-vs-edge and edge-vs-rim separations; near-parallel pairs skipped as in the SAT), the **radial rejection axes** — the component of the center offset perpendicular to each primary axis, which is the true lateral separating direction when two round shapes lie (near-)parallel with an axial offset, exactly where the parallel-axis cross product vanishes and the raw center line is tilted — and the **center line**; the maximum gap is the separation, its axis (oriented $i \rightarrow j$) the normal. **Exact for face-on contacts** — in particular every wall-slab contact in a BOX — and **exact for side-side contacts of parallel and near-parallel round shapes**: two parallel cylinders with overlapping axial ranges separate by exactly the lateral gap, with a purely lateral normal (test `parallel_cylinders_side_contact_is_exact`; this used to fire early with a tilted normal). The conservative caveat now applies **only to genuinely skew corner-on configurations** (contact may be reported slightly early, along a candidate axis); the torus is seen as its convex hull, so only balls can thread the hole. One face-on configuration stays invisible outright: two disks with **parallel** planes both have zero extent along the shared normal, so their separation is $|dz|$ — it touches zero at plane coincidence without a sign change, exactly like the point-through-disk case, and the crossing produces no root for the downward-crossing rootfinder (pinned by test `parallel_disk_disk_separation_is_the_documented_limitation`; tilt one disk or model a thin cylinder — see the FAQ). **Contact point = the support point of the lower-support-rank body** — the incident body’s deepest point (a tilted cylinder’s rim point against a wall face, not the centroid of the wall face). **Equal ranks prefer the body with the clearly smaller flat footprint** (`support_footprint_radius`, the lateral circum-radius of the supporting set: chosen when under half the partner’s), so a small cap landing on a big wall face contacts at the **cap center**, not halfway toward the face centroid (test `small_cap_on_large_face_contacts_at_the_cap_center`); comparable footprints use the midpoint of the two support-set centroids.

Compound decomposition (the dumbbell). A dumbbell never reaches a dispatch tier as a whole: the narrow phase **decomposes dumbbell-vs-anything over its parts**. Each sphere part is a ball served by the exact ball tier — the partner shape’s exact SDF evaluated at the sphere center — which is exact against **every** shape, *including another dumbbell*, whose union SDF is itself exact. The rod recurses as a free-standing cylinder, served with a cylinder’s usual exactness profile (exact face-on and parallel side-side, conservative only on skew corners); contact selection takes the deepest part (the `ball_vs_shape_contact` consolidator plus the pose-level `contact_geometry_at`). So in dumbbell-vs-dumbbell every part pair that involves a sphere is exact, and only **rod-vs-rod** ever reaches the approximate support-axis tier. This part-wise exactness is what the conservation anchor rests on (Sections 8 and 9, Example 12): the impulse pair acts at one shared contact point, so two tumbling dumbbells colliding off-center conserve E , P and L through real CVODE events.

The rigid box (BOX <size>). Six static Cuboid wall slabs enclosing an axis-aligned cube of the

given inner side length, each with `inverse_mass = 0` **and** zero inverse inertia. This is infinite mass *exactly as the equations of motion see it*: the state stores momenta and the dynamics only ever multiply by the inverse — $v = p m^{-1}$, and the impulse denominator is $\hat{n}^\top K \hat{n} = m_i^{-1} + m_j^{-1} +$ angular terms. A static side therefore contributes exactly 0 to every denominator, receives no state writes at all, and reflects everything elastically while remaining bit-identically at rest — no large-mass approximation appears anywhere.

The root function is the narrow phase: the same `separation_at` geometry runs inside the CVODE/ARKODE g function on the packed state, so detection and response can never disagree.

Anti-tunneling with spin — and with acceleration. The while-armed step cap is (smallest crossable feature)/(2 × max *reachable* relative *surface* speed), where *reachable* spans the horizon Δt to the next refresh — one output interval (the cap is re-refreshed at every interval start and after every event). The linear term is the relative center speed **plus** $(a_i + a_j) \Delta t$, each body’s acceleration taken from pairwise gravity at the current configuration, uniform gravity, qE and the external force (the magnetic force is $\perp v$ and can never grow speed) — so a ball released **from rest** above a thin plate, whose instantaneous relative speed is zero at arm time, still gets a finite cap and is caught (test `ball_released_from_rest_does_not_tunnel_the_thin_plate`: TOI matches the analytic $\sqrt{2h/g}$ fall to 10^{-6}). The spin term bounds each body’s rate as $|\omega| \leq \sqrt{3} \|I^{-1}\|_\infty (|L| + \Delta t (|\tau_{\text{ext}}| + \|M\|_\infty |B|))$, multiplied by its R_{bound} — a bound through the angular *momentum* rather than the instantaneous $|\omega|$, so it covers torque-free polhode motion exactly (L is conserved, yet $|\omega|$ can spike up to $|L|/I_{\text{min}}$ mid-interval with no torque at all). The cap returns no-cap only when literally nothing can move. Feature sizes of the new shapes: the torus’s tube radius, the cylinder’s $\min(r, h)$; the ideal disk has zero thickness of its own, so its cap comes from the radius of whatever ball approaches it (the pairwise min picks the smaller feature).

The elastic closing sweep. `resolve_penetrations` now takes an explicit mode: the Zeno tier-2 guard still resolves **plastically** (kill a chattering contact dead), but the end-of-interval safety sweep resolves **elastically** — a slowly-grinding approximate (support-axis) contact that never produces a clean root can no longer bleed energy sweep after sweep.

The hmax-span fix (an honest bug report). The after-event refresh of the anti-tunneling cap clamps the computed `hmax` into a span, and that span used to be $t_{\text{out}} - t$ — the remainder of the current *output interval*. A collision landing exactly **on** an output boundary (which the box geometry of Example 11 promptly produced) collapsed that span to zero, pinning the max step at the 10^{-12} clamp floor; the stale cap was never recomputed, so every later interval was starved into `CV_TOO_MUCH_WORK` (500 000 no-advance steps). The fix is twofold: the clamp span is now the remaining **run** ($t_{\text{end}} - t$, exactly as at arm time), and the cap is re-refreshed at the start of every output interval, so a cap computed near a previous t_{out} can never go stale.

Defaults. restitution 1.0 (elastic), combine $\min(e_i, e_j)$; `restitution_threshold` 10^{-3} ; `contact_slop` 10^{-9} ; Zeno tiers 64 / 128 events per output interval.

7 Command reference

command / path	meaning
COLLIDE	report status: on/off, collidable pairs, impulses so far
COLLIDE ON / COLLIDE OFF	enable (default) / disable detection

CONTACTS	list every contact of the last STEP/RUN
NEW TORUS { ... }	new torus: <code>ring_radius/tube_radius</code> (defaults 1 / 0.25) or the <code>inner_radius + outer_radius</code> pair — resolved and validated once at the closing brace, so the pair is genuinely order-independent; <code>inner_radius = 0</code> (the horn torus) is valid
NEW DISK { ... }	new ideal zero-thickness solid disk: <code>radius</code> (default 1)
NEW CYLINDER { ... }	new cylinder: <code>radius</code> (default 0.5); <code>height</code> sets the <i>full</i> height (default half_height 1)
NEW CUBE / NEW DISC	aliases for CUBOID / DISK
NEW DUMBBELL [AS <name>] { ... }	new rigid two-spheres-plus-rod body: <code>m1, m2, m_rod, r1, r2, rod_radius, length</code> (defaults 1, 1, 0.5, 0.25, 0.25, 0.1, 1) plus the ordinary entity values — deferred and order-independent, validated once at the closing brace via <code>boundary::dumbbell</code> (a failed NEW leaves no ghost); the local origin is placed at the COM; DUMBBELL is an accepted alias spelling; AS <name> registers a user name usable in every path (<code>d.mass, d.vx</code>)
objN.m1 / .m2 / .m_rod	the dumbbell part masses, read and write — the stored mass fractions make every part recoverable, and a member write rebuilds total mass, COM offsets and the inertia tensor in one step
objN.r1 / .r2 / .rod_radius / .length	the dumbbell geometry (<code>rod_r/len</code> accepted), read and write — same one-step rebuild
BOX <size> / BOX OFF / BOX	create / remove / report the rigid bounding box: six static wall slabs with <code>inverse_mass = 0</code> (infinitely massive)
objN.ring_radius, .tube_radius	the torus generating radii (writes recompute inertia)
objN.inner_radius, .outer_radius	the derived torus pair (each write holds the other one fixed)
objN.height / .half_height	cylinder height (<code>height = 2 × half_height</code>)
objN.radius	now serves sphere, disk and cylinder (a write keeps the shape family); a torus write errors — set <code>ring_radius/tube_radius</code> or <code>inner_radius/outer_radius</code> instead
objN.inverse_mass	0 for a wall slab — infinite mass, exactly
GET system.box	inner box size (0 = none)
GET contactK.i / .j	the colliding pair (indices)
GET contactK.t	event time (the interpolated root)
GET contactK.point	world contact point (vec3)
GET contactK.normal	unit action–reaction line , $i \rightarrow j$
GET contactK.depth	penetration depth at the event
GET contactK.rel_vel_n	approach speed along the normal (negative)
GET contactK.impulse	scalar impulse magnitude J

SET objN.restitution = e	bounciness 0 (plastic) ... 1 (elastic, default)
GET system.contacts	number of recorded contacts
GET system.collisions	running impulse total this session
SET	resting-contact speed threshold
system.restitution_threshold	
SET system.contact_slop	tolerated overlap before projection

Contact paths are ordinary expression atoms: `contact0.impulse * contact0.normal.x` computes the x-component of the impulse vector. Machine mode's `{"op": "state"}` includes `"contacts"`, `"collide_enabled"`, `"collision_count"`, and — since this release — `"box"` (number or null) plus per-object `"wall"` and `"inverse_mass"`. The scene window's **Contacts** button / **C** key toggles golden normal arrows at the contact points.

LIST tags each wall slab [`wall: static, inverse_mass=0`] (any other inverse-mass-0 body reads [`static, inverse_mass=0`]); DEL keeps the wall index list renumbered, and deleting any wall dissolves the box (`system.box` → 0) while the surviving slabs **stay tracked**: LIST keeps their [`wall`] tag, bare BOX reports `box: dissolved` (a wall was deleted; N tracked slab(s) remain — BOX `<size>` replaces them, BOX OFF removes them), and BOX `<size>` removes the survivors before building the new box — no orphan slab is ever leaked. Scene init entities carry the new shape parameters and `"wall": true`, with a top-level `"box": <size>`; `scene.html` draws torus (outer/inner equators + tube rings + 4 cross-sections), disk (rim + 2 diameters) and cylinder (2 rims + 4 side lines) wireframes, all quaternion-rotated so spin is visible, plus a dashed #5d84a8 interior wireframe for the box — the wall slabs themselves are **not** drawn as bodies.

8 How it was verified

39 unit tests (geometry: known separations, SAT face vs. edge, continuity across touch; response: $\hat{n} \cdot K \hat{n} = m_1^{-1} + m_2^{-1}$ for central hits, static-wall reflection with the wall bit-unchanged, separating pairs untouched, projection separating deep overlap; new this release: the torus/disk/cylinder **SDFs at hand-checked points**, support extents/points (the invariant $p \cdot u = h(u)$ verified across shapes and directions) and ranks, the new **analytic inertia tensors** (torus $I_z = 2.4375$ for $m = 1$, $c = 1.5$, $a = 0.5$; disk $I_z = I_x + I_y$), ball-vs-torus contact with the hole genuinely passable, disk face/rim and cylinder side/cap contacts, the **tilted-torus anchor** — a flat torus of outer radius 2 exactly inscribes a 4-wide box (separation 0 to 10^{-12}) while the $(1, 1, 1)/\sqrt{3}$ tilt clears every wall by exactly $2 - (1.5\sqrt{2/3} + 0.5) \approx 0.2753$ — and a static slab **reflecting a cylinder elastically** with the wall bit-unchanged; new in the review round: **exact side-side contacts of parallel cylinders** via the radial rejection axes (`parallel_cylinders_side_contact_is_exact` — the lateral gap to 10^{-12} across four offsets and a purely lateral normal), the **pinned parallel disk-disk limitation** (`parallel_disk_disk_separation_is_the_documented_limitation` — separation = $|dz|$ exactly, touching zero without a sign change) and the **smaller-footprint contact rule** (`small_cap_on_large_face_contacts_at_the_cap_center` — a 0.25-radius cap pressed into a big slab face contacts at the cap center)); new in the dumbbell release: `dumbbell_constructor_com_sdf_and_support` — the constructor's COM placement ($m_1 z_1 + m_2 z_2 + m_{\text{rod}}(z_1 + z_2)/2 = 0$ exactly), the union SDF at hand-checked points, the composite inertia, and the **directed** supports ($h(u) \neq h(-u)$ for the off-center spheres; every symmetric shape's extents unchanged) — and `dumbbell_wall_gaps_and_ball_contacts_ar`

— an **asymmetric** dumbbell’s wall gaps exact at **both** ends (the light end pokes farther: $|z_1| > |z_2|$ when $m_2 > m_1$), plus ball-vs-pole and ball-vs-rod contacts exact. 16 integration tests, each against a closed form: velocity exchange with **TOI error** $< 10^{-9}$, unequal-mass formulas, restitution ratios and plastic KE loss $\frac{1}{2}\mu v^2$, oblique normals = line of centers, off-center $\Delta L = r \times J$ with total L conserved, 3-sphere cradle, bounce apex $= e^2 h$ at $t = \sqrt{2h/g}$, **no tunneling** at 100 m/s vs. a 5 mm plate, SPRK-path exchange, **bit-identical zero-pair invariance**, armed-but-quiet agreement $\leq 10^{-12}$; new this release: a **ball in a rigid box** of inverse-mass-0 slabs (energy conserved to 10^{-9} , the ball never escapes, all six walls bit-identical to construction), a **point particle threading the torus hole** with zero events while a fat ball on a nearby line hits the tube at the analytic $\text{TOI} = (3 - \sqrt{1.45})/4$ and the free torus recoils (E and p conserved), and a **mixed cylinder + disk + cube rattle** inside the box with $|dE|/E < 10^{-6}$; new in the review round: a ball **released from rest** above a thin static plate under uniform gravity, caught at the analytic free-fall $\text{TOI} = \sqrt{2 \cdot 4.985/10}$ to 10^{-6} (**ball_released_from_rest_does_not_tunnel_the_thin_plate** — the acceleration-aware anti-tunneling cap at work: the instantaneous relative speed is zero at arm time); new in the dumbbell release, the headline anchor: **two tumbling dumbbells colliding off-center conserve E, P AND L (about the origin) to 10^{-8} through the real CVODE event path** (**colliding_dumbbells_conserve_energy_momentum_and_angular_momentum**) — the impulse pair acts at one shared contact point, so the net torque is zero. Grammar tests (39 posim tests) cover the whole command family including error paths, now with the NEW TORUS/DISK/CYLINDER parameter paths (the order-independent inner/outer pair, HEIGHT as the full height, radius across the shape family) and the whole BOX family (creation, status, OFF, system.box, wall tags, machine-mode box/wall/inverse_mass); new in the review round: **torus_pair_is_order_independent_and_new_is_tra** (both orders of the inner/outer pair — including { **outer_radius** = 0.5, **inner_radius** = 0.2 }, which used to fail in one order — the horn torus **inner_radius** = 0 on NEW and SET, the refused SET **objN.radius** on a torus, and a failed NEW leaving **no ghost object** behind) and **box_recreate_after_wall_deletion_leaks_nothing** (bare BOX reports the dissolved state; BOX **<size>** after a wall deletion removes the surviving tracked slabs first); new in the dumbbell release: **def_call_named_objects_and_dumbbell_members** — the full **create_dumbbell** flow (define / call / member reads and writes / shorthands / renumber / errors / redefine) — and the scene test **reset_restores_the_initial_state_and_start_reruns**. The scene frame was verified live to carry the exact analytic normal/point/impulse, and the arrow’s pixels were verified on the canvas; for this release the box-of-shapes demo was re-verified in a real browser: the **init** frame carried **box: 4**, the wall flags and every new shape’s parameters; the window’s playback copy conserved E to 10^{-9} ; the golden contact arrows drew (2218 gold pixels counted on the canvas) and the dashed box wireframe drew (5904 pixels); reverse playback was exercised. All regression anchors (outer solar system, Kepler, gyroradius, tumbling body) still print **SUCCESS**; **104 tests green** (40 lib + 16 collision + 9 conservation + 39 posim); the workspace builds with zero warnings; **Cargo.lock** still lists only the five local crates.

9 The twelve examples (run for the USER and the MANAGER)

All scripts live in `scripts/collisions/` and run with

```
cargo run -p posim - --script scripts/collisions/NN_name.posim.
```

The outputs below are the real captured runs. Two additional self-checking Rust examples (`newtons_cradle`, `bouncing_ball_restitution`) print **SUCCESS/FAILURE** and exit nonzero on failure.

9.1 Example 1 — head-on exchange (the canonical action–reaction pair)

Two identical spheres approach at ± 1 ; gap 3, closing speed 2 $\Rightarrow$ impact at $t = 1.5$ exactly; velocities exchange, momentum stays 0, energy is conserved.

```
Out[7]= t = 3 (advanced by 3, 26 solver steps, 1 collision(s) - CONTACTS lists them)
Out[8]= contact0: obj0 <-> obj1 at t = 1.5
      point = [0, 0, 0]
      normal = [1, 0, 0] (from obj0 toward obj1)
      depth = 0, approach speed = 2, impulse = 2
      get obj0.velocity -> [-1, 0, 0]      get obj1.velocity -> [1, 0, 0]
      energy -> 1 (unchanged)
```

9.2 Example 2 — unequal masses (the light ball bounces back)

$m_1 = 1$ at $v = 1$ strikes $m_2 = 3$ at rest: $v'_1 = (m_1 - m_2)/(m_1 + m_2) = -0.5$, $v'_2 = 2m_1/(m_1 + m_2) = +0.5$.

```
get obj0.velocity.x -> -0.5      get obj1.velocity.x -> 0.5
momentum -> [1, 0, 0] (conserved)
```

9.3 Example 3 — the restitution ladder

The same collision with $e \in \{1.0, 0.8, 0.5, 0.2\}$: separation speed = $e \times 2$ and kinetic energy = e^2 in every rung.

e = 1.0:	separation 2	energy 1
e = 0.8:	separation 1.6	energy 0.64
e = 0.5:	separation 1	energy 0.25
e = 0.2:	separation 0.4	energy 0.04

9.4 Example 4 — Newton's cradle (five spheres)

Four touching balls, one incomer at $v = 1$; the Gauss–Seidel passes propagate the impulse through the simultaneous contacts:

```
t = 4 (... , 4 collision(s))
v0..v3 -> 0, 0, 0, 0      v4 -> 1      momentum -> [1, 0, 0]
```

Only the far ball exits, at the incomer's speed.

9.5 Example 5 — billiard break (every outgoing direction is a normal)

A cue ball at $v = 2$ breaks a three-ball triangle at $e = 0.95$:

```
contact0: normal [1, 0, 0]      (cue -> apex, head on)
contact1: normal [0.835..., 0.55, 0] (apex -> upper ball)
contact2: normal [0.835..., -0.55, 0] (apex -> lower ball)
momentum before -> [2, 0, 0]    after -> [2, 0, 0] (exact)
```

9.6 Example 6 — off-center spin-up ($L = r \times J\hat{n}$)

A sphere strikes a free cuboid above its center; contact at $[-0.5, 0.5, 0]$ with $J = 4.444\dots$ along $+x$:

```
get obj1.angular_momentum -> [0, 0, -2.2222222222222223]
```

$r \times J\hat{n} = (0, 0, -0.5 \cdot 4.444) = (0, 0, -2.222) \checkmark$; total angular momentum about the origin stays -2 through the transfer.

9.7 Example 7 — the thin-wall time of impact (the tunneling killer)

A 1 cm bullet at speed 100 meets a 5 mm static plate inside a single 0.05 s output step; analytic $\text{TOI} = (2 - 0.005 - 0.01)/100 = 0.01985$:

```
get contact0.t      -> 0.0198500000000000846   (error ~ 8e-16!)
get obj1.velocity.x -> -100                    (reflected exactly)
get obj1.position.x -> -3.03                   (back on its own side)
```

A fixed-step engine would tunnel; the integrator's root finder cannot miss.

9.8 Example 8 — colliding gravitational binary

Two spheres on a bound orbit whose pericenter (0.5) is below the touching distance (0.6), $e = 0.6$; the event fires on the *curved* trajectory:

```
contact0 at t = 2.884... normal [-0.556, 0.831, 0] (line of centers at TOI)
energy -0.4 -> -0.4996 (the inelastic bounce shrinks the orbit)
momentum -> [2.8e-16, -2.2e-16, 0] (zero to rounding)
```

9.9 Example 9 — the spinning target ($\omega \times r$ terms at work)

The cuboid tumbles at $\omega_z = 2$; the struck face carries its own velocity:

```
contact0: normal [-0.986, 0.167, 0] (the ROTATED face normal)
angmom before -> [0, 0, 0.16] after -> [0, 0, 0.15999999999999956]
energy before -> 2.96 after -> 2.9599999999999995
```

Angular momentum and energy conserved through a rotating-surface impact.

9.10 Example 10 — the billiard box (seven bounces, zero drift)

An elastic ball ping-pongs between two static walls for 20 time units:

```
t = 20 (356 solver steps, ..., |dE/E| = 0.000e0, 7 collision(s) ...)
get system.collisions -> 7 energy -> 1.4449999999999998 (unchanged)
get obj2.velocity.x -> -1.7
```

Also the scene demo: **SCENE CREATE + SCENE START** shows a golden normal arrow at every wall strike.

9.11 Example 11 — the box of shapes (every shape in the infinitely massive box)

$R = 1$, $M = 1$. BOX 4 builds the rigid box; six bodies fly inside it: a **torus** M (inner radius 1, outer radius 2), a **point** M at $v = (100, 200, 100)$ — the only mover, so $E_0 = \frac{1}{2} \cdot 1 \cdot 60000 = 30000$ exactly — a **sphere** $2M$ of $r = 1/2$ (the spec leaves the radius free; $R/2$ is documented in the script), a tilted **disk** $2M/3$ of $r = 1$, a **cube** $5M/3$ of side 1, and a tilted **cylinder** $2M$ of $r = 1/2$, height $3/2$. The geometry is sharp: an axis-aligned torus of outer radius 2 **exactly inscribes** the 4-box (its outer equator touches four walls), so “inside and not touching” forces a tilt — with the axis along the body diagonal $(1, 1, 1)/\sqrt{3}$ the extent per axis is $1.5\sqrt{2/3} + 0.5 \approx 1.7247 < 2$ (clearance 0.2753) and the torus sits at the center. The other five positions are drawn by a documented LCG ($x \leftarrow 1664525x + 1013904223 \bmod 2^{32}$, seed 20260724, sequential rejection at ≥ 0.05 separation), so the script is reproducible, not hand-tuned.

```
In[2]:= box 4
Out[2]= box: inner size 4 x 4 x 4 -- six static walls obj0, obj1, obj2, obj3, obj4,
        obj5 with inverse_mass = 0 (infinitely massive); objects collide elastically off
        the inside faces
In[10]:= collide
Out[10]= collisions ON (51 collidable pair(s); 0 impulse(s) so far)
get obj0.inverse_mass -> 0
get obj6.inertia_tensor -> [[1.28125, 0, 0], [0, 1.28125, 0], [0, 0, 2.4375]]
energy -> 30000          momentum -> [100, 200, 100]
In[15]:= run 0.1 steps 100
Out[15]= t = 0.1 (2121 solver steps, 100 snapshots, |dE/E| = 2.040e-10, 119 collision(s)
        ) -- CONTACTS lists them)
energy -> 29999.999993880127
momentum -> [146.48911126803657, 102.1478121131382, 46.01601291636828]
```

119 impulses across all three dispatch tiers in 0.1 time units, and the energy drift is $|dE/E| = 2.040 \cdot 10^{-10}$. The 51 collidable pairs are the 66 pairs of 12 bodies minus the 15 static wall-wall pairs; the torus inertia readback matches the analytic tensor exactly ($m(\frac{1}{2}c^2 + \frac{5}{8}a^2) = 1.28125$, $m(c^2 + \frac{3}{4}a^2) = 2.4375$ for $c = 1.5$, $a = 0.5$). **Momentum is NOT conserved** — $(100, 200, 100) \rightarrow (146.49, 102.15, 46.02)$ — **and is not supposed to be**: every wall impulse transfers $\pm J\hat{n}$ into a body with `inverse_mass = 0`. The infinitely massive walls absorb momentum at zero velocity, so they absorb no energy ($p^2/2m \rightarrow 0$ as $m \rightarrow \infty$, taken *exactly* by the inverse-mass formulation), and after 119 collisions the six walls are **bit-identically at rest**. Momentum non-conservation with exact energy conservation is the physical signature of a rigid container — not a bug.

9.12 Example 12 — two dumbbells from a user-defined function — E , P and L conserved

`create_dumbell` is defined **in the notebook language** (DEF): it takes a name plus every entity value — all with defaults — and builds a named rigid dumbbell (two solid spheres joined by a solid rod, one rigid body whose local origin is its center of mass). Called twice, it makes `dumbell0` (sphere masses 1 + 2, rod 0.5) and `dumbell1` (2 + 1 + 0.4, different radii and length); they approach off-center with spin and collide **twice** ($G = 0$, restitution 1, no walls). The captured run (`12_two_dumbbells.posim`; abridged — the `.md` twin has every line):

```

In[2]:= def create_dumbbell(name, m1 = 1, m2 = 1, m_rod = 0.5, r1 = 0.25, r2 = 0.25,
    rod_radius = 0.1, length = 1, position = [0, 0, 0], velocity = [0, 0, 0],
    angular_velocity = [0, 0, 0]) {
    new dumbbell as name { m1 = m1, m2 = m2, m_rod = m_rod, r1 = r1, r2 = r2, rod_radius
    = rod_radius, length = length, position = position, velocity = velocity,
    angular_velocity = angular_velocity }
}
Out[2]= function create_dumbbell(11 parameter(s)) defined -- 1 body line(s)
In[3]:= create_dumbbell("dumbell0", 1, 2, 0.5, 0.25, 0.25, 0.1, 1, [-2, 0.15, 0], [1.5,
    0, 0], [0, 0, 0.6])
Out[3]= obj0 as dumbell0
In[4]:= create_dumbbell("dumbell1", 2, 1, 0.4, 0.3, 0.2, 0.08, 1.2, [2, -0.15, 0],
    [-1.5, 0, 0], [0.4, 0, 0])
Out[4]= obj1 as dumbell1
get dumbell0.m1          -> 1
get dumbell1.m_rod       -> 0.39999999999999999
get dumbell0.vx          -> 1.5
energy                   -> 7.865310611764706
momentum                 -> [0.150000000000000036, 0, 0]
angmom                   -> [0.4443030588235295, 0, -1.5059999999999998]
In[12]:= run 3 steps 60
Out[12]= t = 3 (3861 solver steps, 60 snapshots, |dE/E| = 9.463e-11, 2 collision(s) --
    CONTACTS lists them)
energy                   -> 7.865310611020375
momentum                 -> [0.150000000000000036, 0, 0]
angmom                   -> [0.44430305882373156, -0.000000000000009547918011776346,
    -1.5059999999999855]
get system.collisions    -> 2

```

The named member paths are at work in the three `get` lines — `dumbell1.m_rod` reads back 0.39999999999999999 because the part masses are *recovered* from the stored mass fractions ($m_{\text{rod}} = (1 - f_1 - f_2) M$), honest floating point. Through the two collisions: energy 7.865310611764706 $\rightarrow$ 7.865310611020375 (the run line itself prints $|dE/E| = 9.463 \times 10^{-11}$); momentum [0.150000000000000036, 0, 0] **bit-identical**; angular momentum about the origin [0.4443030588235295, 0, -1.5059999999999998] $\rightarrow$ [0.44430305882373156, -9.5 $\times 10^{-15}$, -1.5059999999999855] — drift at the 10^{-13} level.

Why is L conserved here when Example 11 lost momentum? Nothing in this scene is static, and each impulse pair $\pm J\hat{n}$ acts at **one shared contact point**: the two angular impulses about any fixed origin are $x_c \times (+J\hat{n})$ and $x_c \times (-J\hat{n})$ — they cancel exactly. Action–reaction at a single point produces **zero net torque**, and with $G = 0$ there is no other torque source, so total L must survive — and it does, through real CVODE events, entirely from the part-decomposed contact geometry of Section 6 (this is the anchor test `colliding_dumbbells_conserve_energy_momentum_and_angular_mor` which pins E , P and L to 10^{-8}). Verified live in a real browser as well: the entity labels show the user names `dumbell0/dumbell1`, and the window’s labeled conserved-quantities readout displayed $E = 7.86531061$, $P = [0.15000, 0.00000, 0.00000]$ $|.| = 0.15000$, $L = [0.44430, 0.00000, -1.50600]$ $|.| = 1.57017$ identically before and after the impact (after it, L ’s y component read -1.31228 $\times 10^{-13}$).

10 FAQ

Do I have to turn anything on? No — collisions are on by default in every scene; COLLIDE OFF disables, COLLIDE reports.

How precise is the impact time? Interpolated onto the root: measured errors at the 10^{-15} relative level (Examples 1 and 7).

Three bodies touching at once? The solver sweeps all pairs until nothing still approaches (Example 4).

Can objects still tunnel? Not on the CVODE paths (step capped against the smallest feature, and the cap counts speed the accelerations can build before the next refresh — even a from-rest drop is caught). On SPRK the sampling bound is the fixed dt — choose it below (thinnest body)/(fastest approach).

Why is momentum not conserved in the box? Because the walls are infinitely massive, and infinite mass absorbs momentum without absorbing energy: a wall receiving impulse J changes its momentum by J but its kinetic energy by $p^2/2m \rightarrow 0$ as $m \rightarrow \infty$. Ordinarily that is an approximation; here the limit is *exact*, because the equations of motion only ever use the **inverse** mass — $v = pm^{-1}$, impulse denominator $m_i^{-1} + m_j^{-1}$ + angular terms — and a wall with **inverse_mass** = 0 contributes zero to every denominator and receives no state writes at all. Newton’s third law still acts at every contact; the “missing” momentum went into the box (Example 11: (100, 200, 100) $\rightarrow$ (146.49, 102.15, 46.02) while energy is conserved to $2.040 \cdot 10^{-10}$ and the walls stay bit-identically at rest).

Can anything pass through the torus hole? Balls can. Ball-vs-anything uses the exact SDF, and on a line through the hole a small ball’s separation dips toward (inner radius – ball radius) without ever reaching zero — no event fires. Verified by test: a point particle threads the hole with zero collisions while a fat ball on a nearby line hits the tube at the analytic TOI = $(3 - \sqrt{1.45})/4$. Extended shapes (cuboid, disk, cylinder, another torus) are served by the support-axis tier, which sees the torus’s **convex hull** — a support function cannot see a hole — so only balls can thread it.

Why does the point particle pass through the disk? The disk is *ideal*: a zero-thickness solid disk whose “SDF” is the unsigned distance to it — continuous, and zero exactly on the disk. Against a zero-radius point the separation touches zero for a single instant and rises again without ever becoming negative: there is no sign change for the root finder and no overlap to resolve — a measure-zero contact carrying no impulse, which is the exact answer for these two idealized bodies. Give either party finite extent **along the contact normal** — any sphere radius, a cuboid, a cylinder — and the separation genuinely crosses zero and the contact fires (every finite-size body in Example 11 bounces off the disk). The one pairing with zero extent on *both* sides is **two disks with parallel planes**: their separation is $|dz|$, which touches zero at plane coincidence without a sign change, so a face-on disk-disk crossing is exactly as invisible as the point-through-disk case (pinned by test `parallel_disk_disk_separation_is_the_documented_limitation`). Workaround: tilt one disk (a tilted disk has finite extent along the other’s normal, so the separation genuinely crosses zero) or model a thin cylinder.

How do compound bodies collide? The dumbbell — the one compound shape — is **decomposed over its parts** by the narrow phase: each sphere part is a ball tested through the other shape’s exact SDF (exact against everything, *including another dumbbell*, whose union SDF is

exact), the rod recurses as a free-standing cylinder, and the deepest part supplies the contact. Only rod-vs-rod ever reaches the approximate support-axis tier. The parts are geometry only — dynamically the dumbbell stays **one** rigid body (one momentum, one angular momentum, one composite inertia tensor about its COM-origin), which is why two tumbling dumbbells conserve E , P and L through real events (Example 12).

Friction? Designed (per-object μ , Coulomb cone clamp) but deliberately not shipped in this delivery; the exact normal and the action–reaction pair — the stated requirements — are fully delivered, and adding tangential impulses changes no data structures.

Hand-rolled integration anywhere? No. Free flight is SUNDIALS exclusively; an impulse is an instantaneous momentum update at an event boundary — the standard treatment in every engine surveyed.